

Spring MVC



**UP ASI
Bureau E204**

Plan du Cours

- Spring MVC (Définition + Spring web)
- Serveur web vs. Serveur d'application
- Les architectures physiques et logiques
- Spring MVC + Postman
- Postman
- Dépendance web
- Cycle de Vie d'une requête HTTP (Spring Boot+Postman)
- RestController
- TP Spring Boot + Spring Data JPA + Spring MVC (REST) + Postman
- Swagger

Spring WEB

- Plusieurs Projets Spring permettent d'implémenter des applications Web :
 - Framework Spring (qui contient Spring MVC)
 - Spring Web Flow (Implémenter les navigations Stateful).
 - Spring mobile (Détecter le type de l'appareil connecté).
 - Spring Social (Facebook, Twitter, LinkedIn).
 - ...

Nous allons nous intéresser à **Spring MVC**.

SPRING MVC

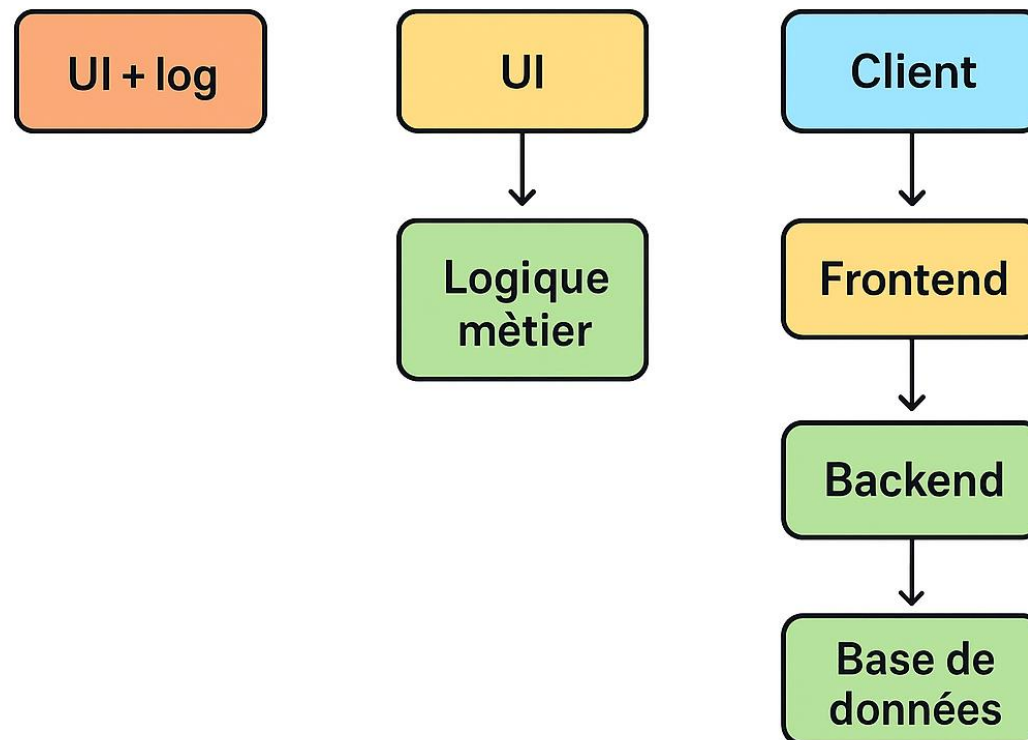
- **Spring MVC** REST permet de créer des API web en utilisant le pattern Model-View-Controller. **Les API REST (Representational State Transfer) utilisent les verbes HTTP pour effectuer des opérations CRUD.**
- **Spring MVC** est un Framework Web basé sur le design pattern **MVC** (Model / View / Controller).
- Spring MVC fait partie du projet “Spring Framework”.
- Spring MVC s'intègre avec les différentes technologies de vue tel que JSF, JSP, Velocity, Thymeleaf...

SPRING MVC

- Spring MVC n'offre pas une technologie de vue mais permet en revanche de communiquer avec toutes les technologies web les plus performantes tels que Angular, React, etc...
- Spring MVC est construit en se basant sur la spécification JavaEE : **Java Servlet**.

Architecture Physique

- **Tier** est un mot anglais qui signifie étage ou niveau.
- Une application peut être **1-Tier**, **2-Tiers**, **3-Tiers** ou **N-Tiers**.

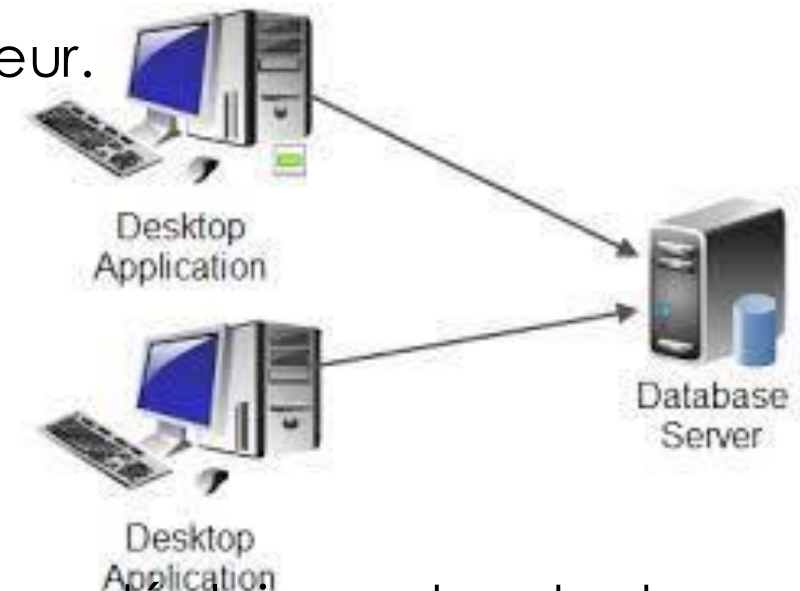


Architecture Physique - 1-Tier

- Une application **1-Tier** est, par exemple, la Modification d'un document Word sur un ordinateur Local.
- Tout est sur la même machine et les couches sont fortement liées.
- Inconvénients : Risque de perte des données (non sauvegardées à distance), Impossible d'accéder à une même ressource par deux utilisateurs en même temps.

Architecture Physique - 2-Tiers

- Une application **2-Tiers** est typiquement une application **client lourd**.
- Le niveau **Présentation (IHM)** et le niveau **Traitement** sont sur la machine de l'utilisateur.
- Le niveau **Base de Données** est sur un autre serveur.
- C'est une architecture **Client / Serveur**.
- Client = demandeur de ressource
- Serveur = fournisseur de ressource



Inconvénients

- Toute mise à jour des fonctionnalités nécessitent un déploiement sur toutes les machines des utilisateurs.
- Le serveur ne fait pas appel à une autre application pour fournir le service.

Architecture Physique - 3-Tiers

- Une application **3-Tiers** introduit un niveau intermédiaire (middleware) entre le client et le serveur.
- Le niveau intermédiaire est chargé de fournir la ressource en faisant appel à un autre serveur.

Avantages

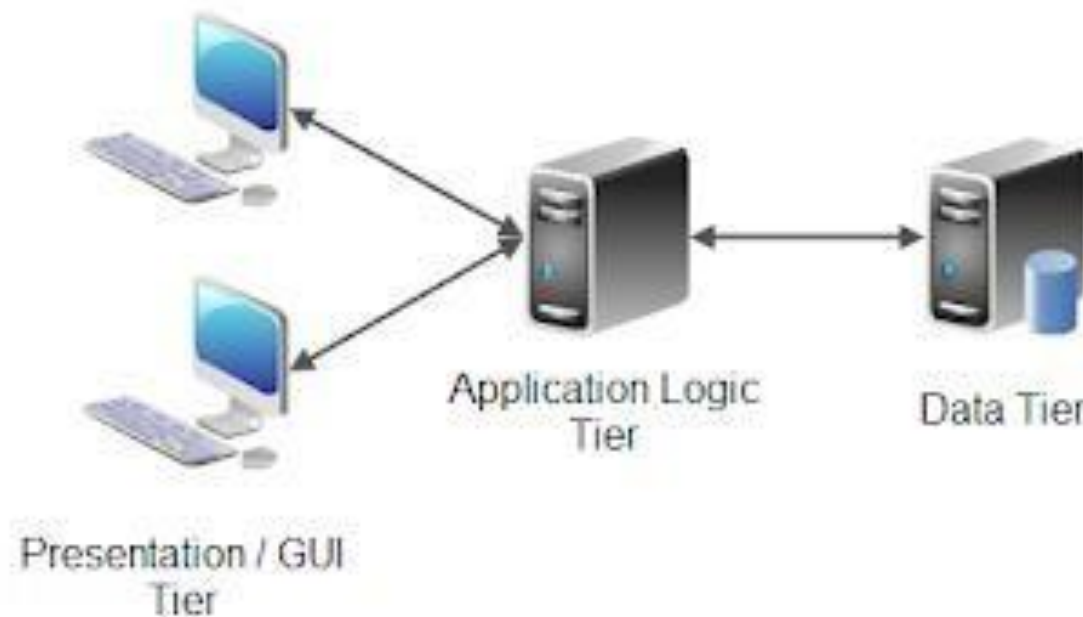
- Centraliser la logique application sur un serveur HTTP

Inconvénients

- Le serveur HTTP (élément principale de l'architecture)est fortement sollicité d'où une charge de demandes provenant à la fois du client et du serveur.
- Bien que cette architecture résout le problème du client lourd de l'architecture deux tiers, le soulagement du client est remplacé par un serveur fortement sollicité.

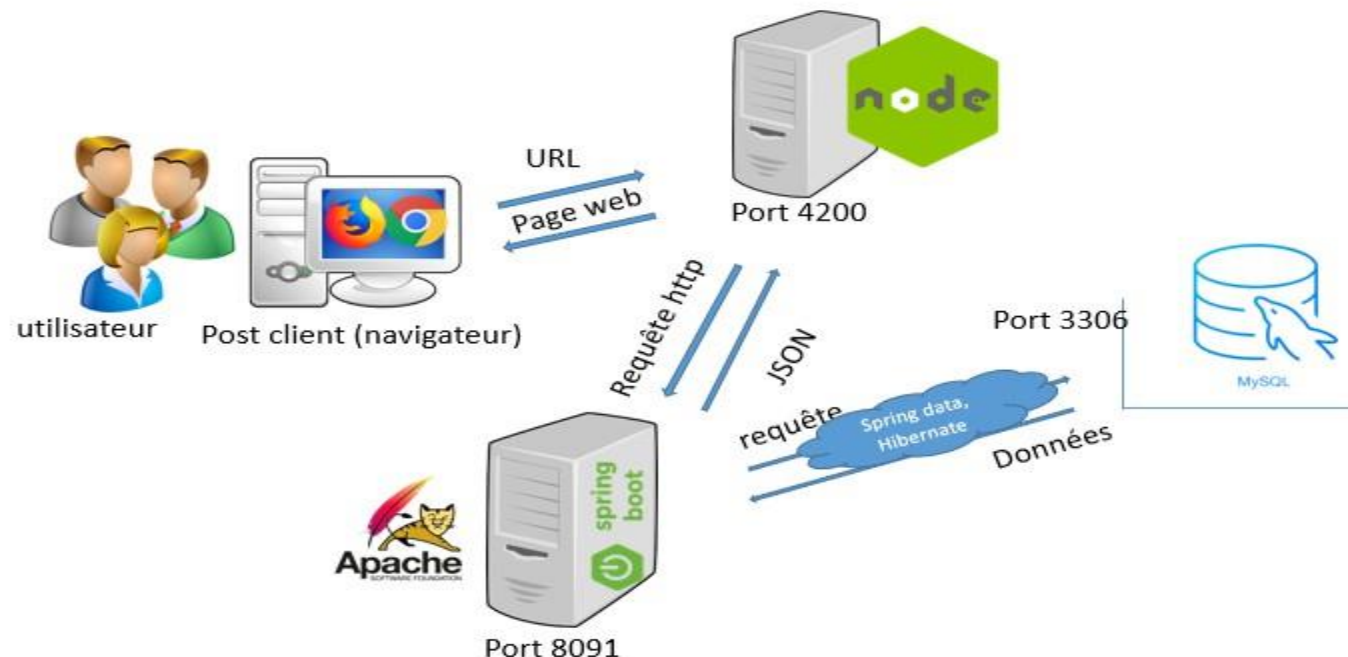
Architecture Physique - 3-Tiers

- Une application **3-Tiers** est typiquement une application Web :
 - Niveau **Présentation** : IHM (Navigateur sur la machine de l'utilisateur)
 - Niveau **Traitement** : Un serveur web (Tomcat, ...) qui contient le WAR de notre application.
 - Niveau **Base de données** : Un serveur de BD qui stocke les données de notre application.

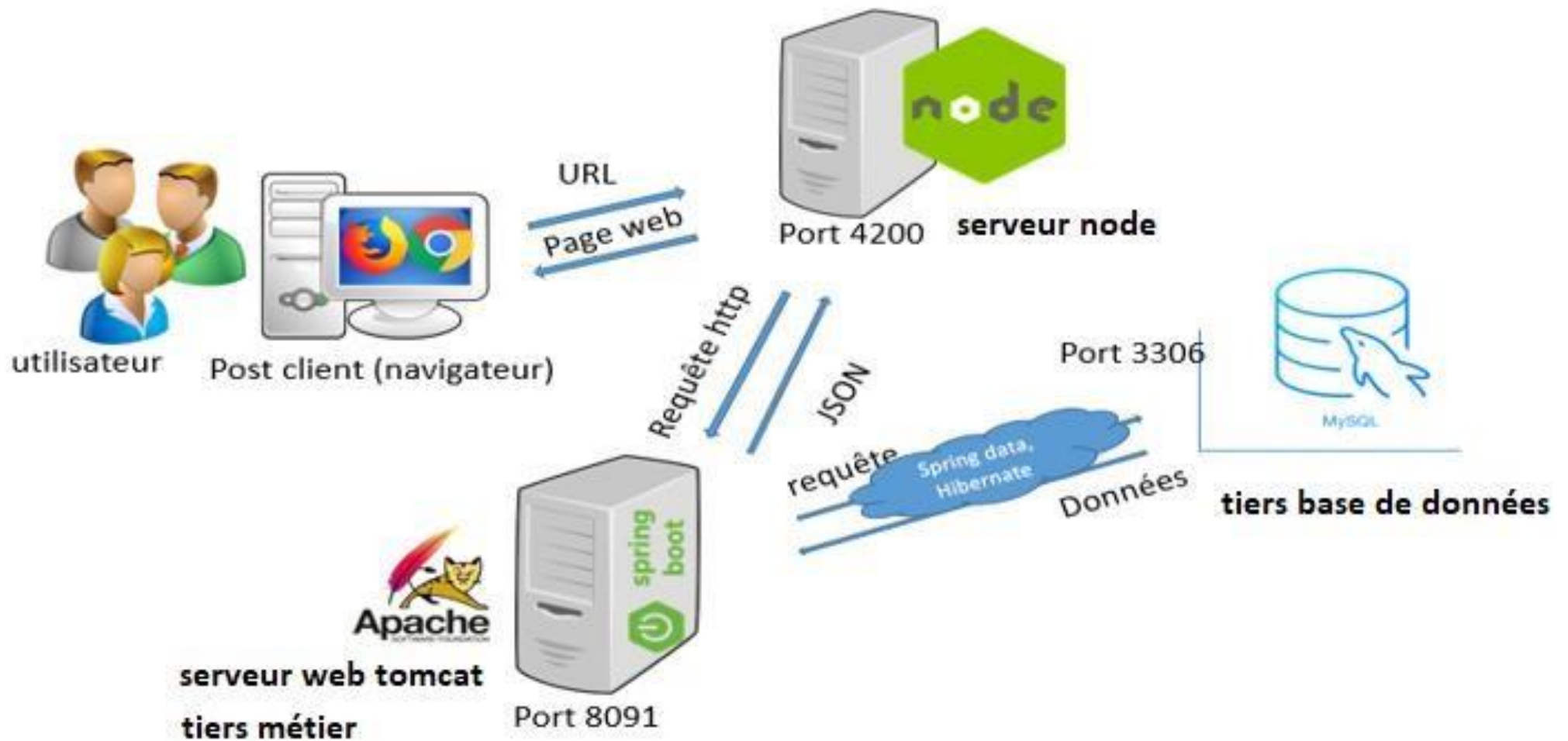


Architecture Physique - N-Tiers

- L'architecture N tiers assure un équilibre de charge entre le client et le serveur par l'introduction de nouvelles couches.
- Voici une architecture 4-Tiers d'une application web développée par un étudiant Esprit pendant son projet de fin d'étude (GUI – Angular sur le Serveur NodeJS – Spring Boot (Serveur Web Tomcat embarqué) – Serveur de base de données MySQL) :

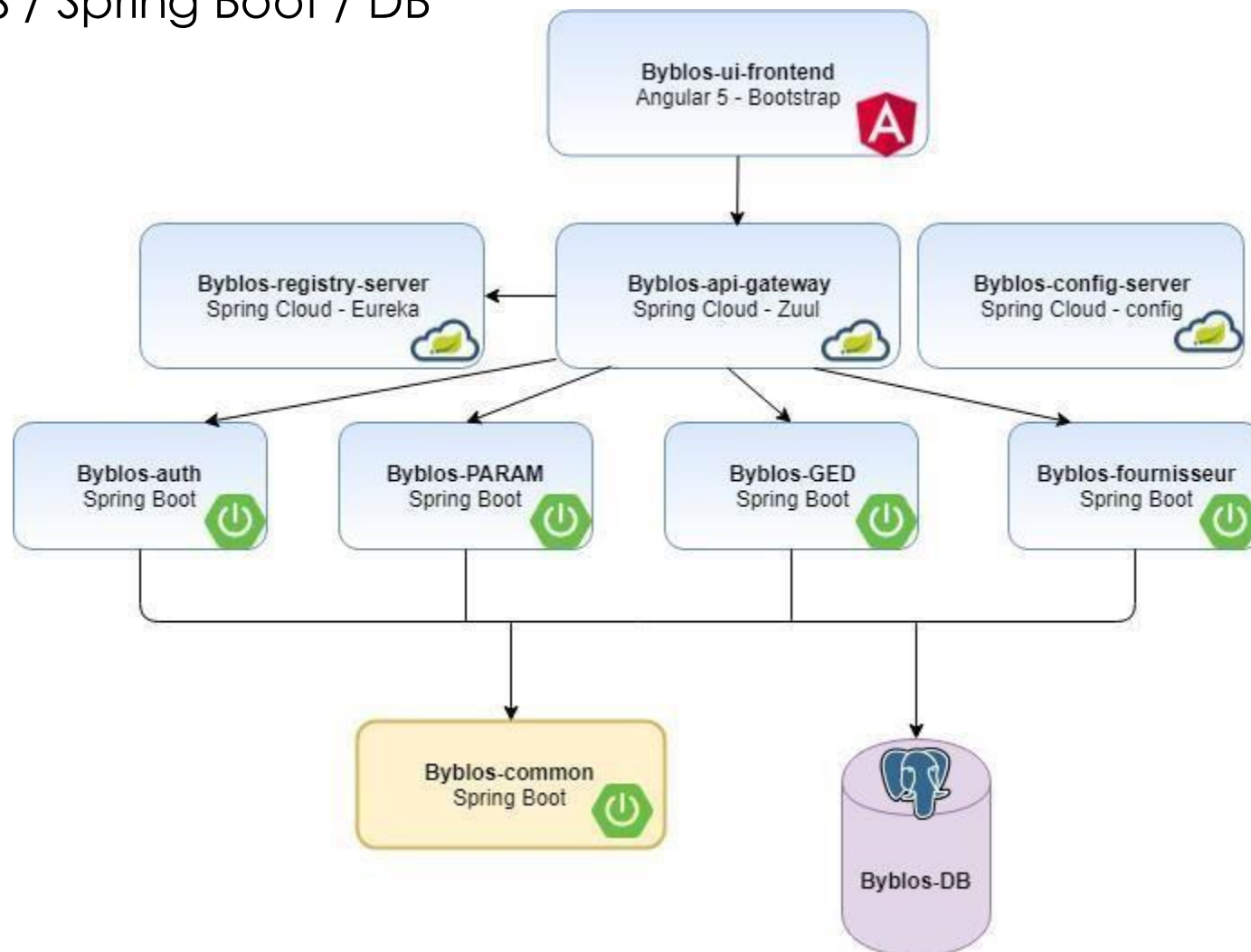


Architecture Physique - N-Tiers



Architecture Physique - N-Tiers

- Voici une architecture n-Tiers, **en Micro-Servcies**, d'une application web développée par un étudiant Esprit pendant son projet de fin d'étude : GUI / NodeJS / Spring Boot / DB

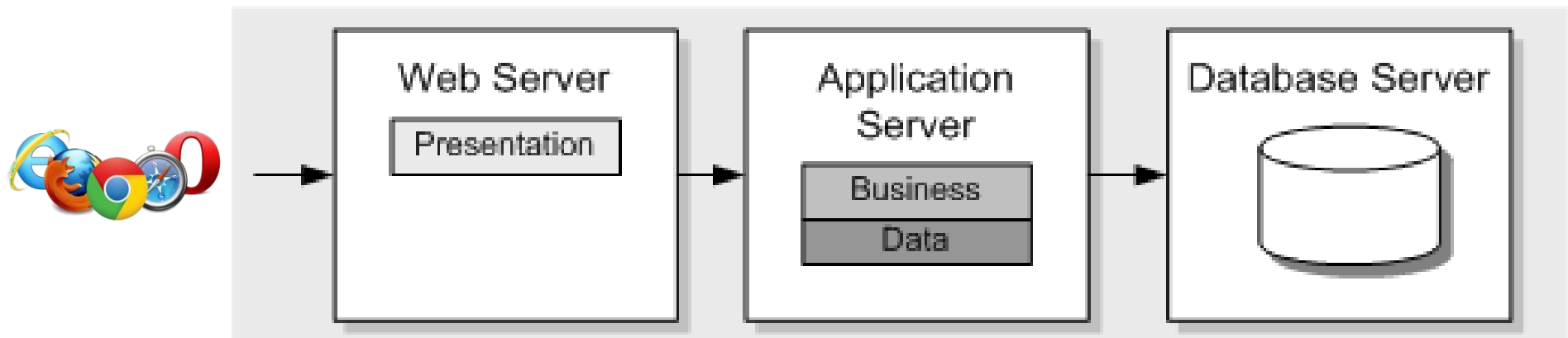


Architecture logique

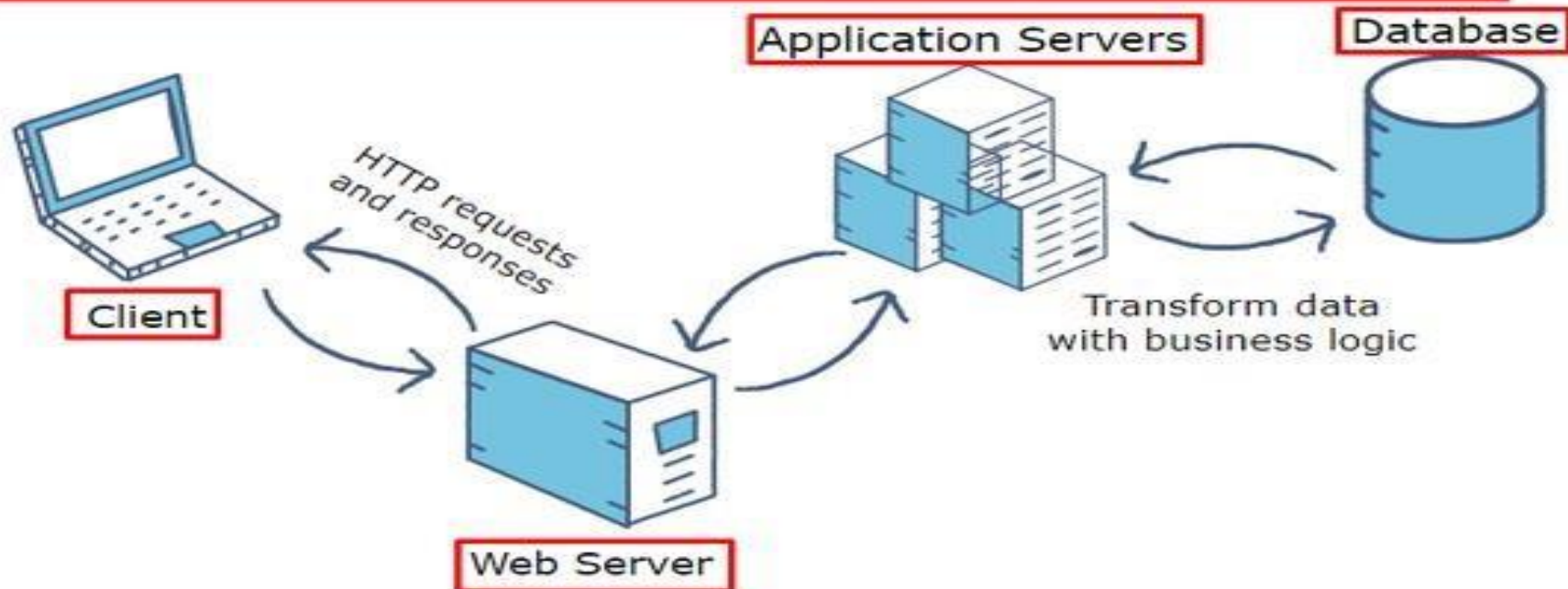
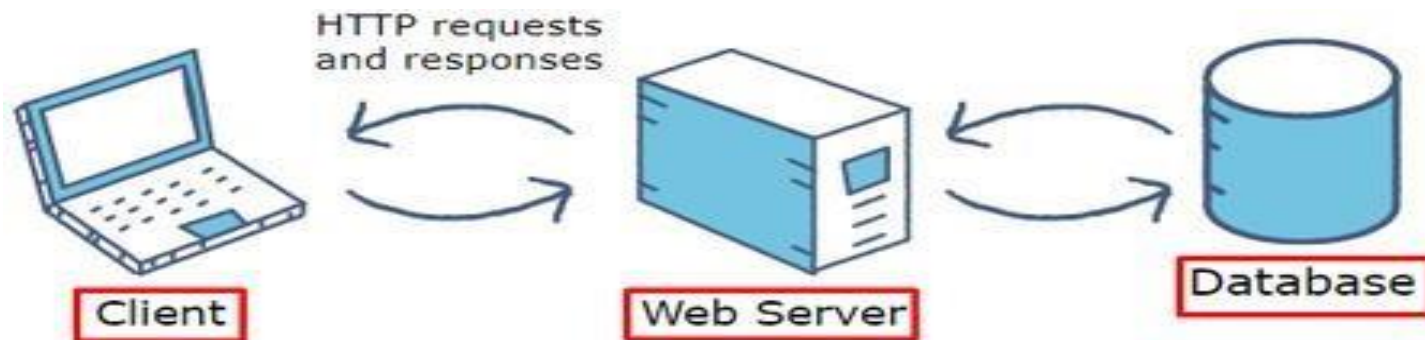
- Une application typique utilisant Spring est généralement structurée en trois couches :
 - Couche **Présentation** : (Web + Contrôleur)
 - Couche **Service** : interface métier avec mise en œuvre de certaines fonctionnalités.
 - Couche **Accès aux Données** : recherche et persistance des objets.
- **Spring est un Framework utilisé pour créer et injecter les objets requis pour communiquer entre les différentes couches.**

Serveur Web vs Serveur d'Application

Serveur Web	Serveur d'application JavaEE Serveur web + container
Héberge que la couche présentation et l'expose qu'à travers le protocole HTTP(S)	Héberge la logique métier et peut aussi héberger la couche présentation (supporte différents protocoles : HTTP, JNDI, ...).
Ne peut pas inclure un EJB Container.	Doit inclure un EJB Container.
lightweight	Relativement gourmand en ressources (CPU, RAM et Disk).
Exp : Apache HTTP Server, Tomcat, Jetty	Exp : Wildfly, WebSphere ...

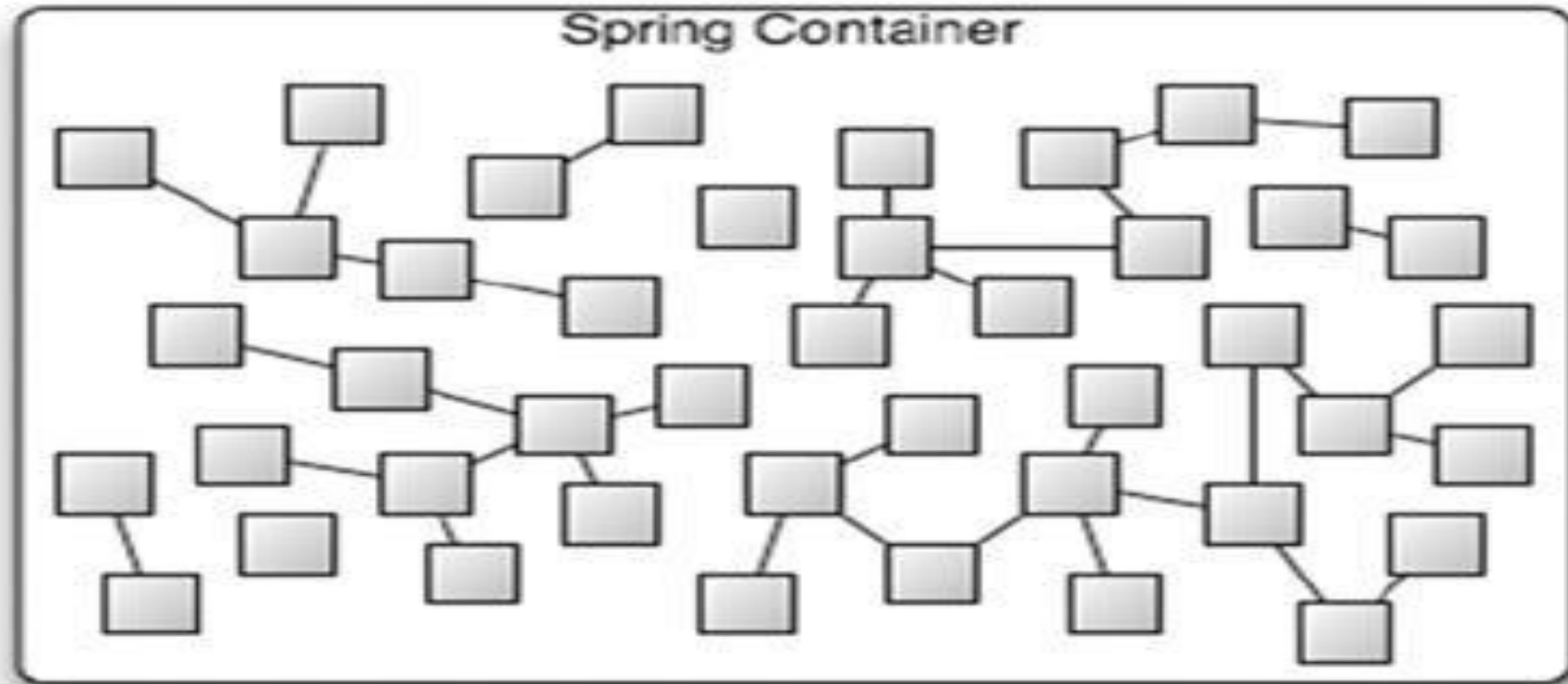


Serveur Web vs Serveur d'Application



Serveur Web vs Serveur d'Application

Spring IOC Container



Dans une application Spring, les objets sont créés, sont liés ensemble et communiquent dans le Spring IOC Container.

Spring MVC et REST (Url de notre Application Web)

- Dans ce fichier de propriétés ajouter les lignes suivantes, pour définir l'url de notre application :

#http://localhost:8081/SpringMVC/sayHello?myName=Walid

#Server configuration

server.port=8081

server.servlet.context-path=/SpringMVC

Pour personnaliser le chemin

Spring MVC et REST (Concepts de base)

Le RestController :

`@RestController` $\equiv$ `@Controller ou @Component`
`et @ResponseBody`

```
public class UserRestControllerImpl {  
  
    @Autowired  
    IUserService userService;  
  
    @GetMapping("/retrieve-all-users")  
    public List<User> getUsers() {  
        List<User> list = userService.retrieveAllUsers();  
        return list;  
    }  
}
```

Spring MVC et REST (Concepts de base)

- **@RestController**: C'est une version spécialisée du contrôleur. Il inclut les annotations **@ Controller** et **@.ResponseBody** (**Indiquer que le type de retour doit être écrit directement dans le corps de la réponse HTTP**) et simplifie donc la mise en œuvre du contrôleur.
- **@GetMapping** pour dire que je vais faire une méthode de lecture.
- **@PostMapping** pour les méthodes d'insertion.
- **@PutMapping** pour les méthodes de modification.
- **@DeleteMapping** pour les méthodes de suppression.

Spring MVC et REST (Concepts de base)

- **@PathVariable** pour obtenir un espace réservé à partir de l'URI.

```
@GetMapping("/api/employees/{id}")
```

```
public String getEmployeeById(@PathVariable String id) { return "ID: " + id; }
```

- **@RequestParam** pour obtenir un paramètre à partir de l'URI.

```
@GetMapping("/api/foos")
```

```
public String getFoos(@RequestParam String id) { return "ID: " + id; }
```

- **@RequestBody** pour convertir le corps de la requête Http à un objet de transfert ou de domaine, ce qui permet la désérialisation automatique du corps HttpRequest entrant sur un objet Java.

```
@PostMapping("/api/employees/add")
```

```
public Employee addEmployee(@RequestBody Employee e) { return er.save(e); }
```

Spring MVC et REST (Postman)

- Parmi les nombreuses solutions pour interroger ou tester les web services et les API, Postman propose de nombreuses fonctionnalités, une prise en main rapide et une interface graphique agréable.
- Postman permet de construire et d'exécuter des requêtes HTTP, de les stocker dans un historique afin de pouvoir les rejouer.



Spring MVC et REST (Postman)

The screenshot shows the Postman interface for a PUT request. The URL is `http://localhost:8089/SpringMVC/servlet/modify-client`. The request body is a JSON object with client details. The response status is 200 OK, and the response body is a JSON object with the updated client data.

Request:

```
PUT http://localhost:8089/SpringMVC/servlet/modify-client
```

Body (JSON):

```
{  2  ... "nom": "Nasri",
  3  ... "prenom": "Ahmed",
  4  ... "dateNaissance": "1995-05-04",
  5  ... "email": "nasri.ahmed@gmail.tn",
  6  ... "password": "pwd1",
  7  ... "profession": "Ingenieur",
  8  ... "categorieClient": "Fidele"
  9  }
```

Response:

```
Body Cookies Headers (5) Test Results
```

Status: 200 OK Time: 903 ms Size: 358 B Save Response

Body (JSON):

```
1  {
2    "idClient": 1,
3    "nom": "Nasri",
4    "prenom": "Ahmed",
5    "dateNaissance": "1995-05-04T00:00:00.000+00:00",
6    "email": "nasri.ahmed@gmail.tn",
7    "password": "pwd1",
```

TP - Spring : Boot – Core – Data JPA – MVC (REST)

- Vérifier que le fichier de propriétés contient les propriétés nécessaires (web, base de données, slf4J, ...) :

`#Server configuration`

`server.servlet.context-path=/SpringMVC`

`server.port=8089`

`### DATABASE ###`

`spring.datasource.url=jdbc:mysql://localhost:3306/springDB?useUnicode=true
&useJDBCCompliantTimezoneShift=true&useLegacyDatetimeCode=false&serverTimezone=UTC`

`spring.datasource.username=root`

`spring.datasource.password=`

`### JPA / HIBERNATE ###`

`spring.jpa.show-sql=true`

`spring.jpa.hibernate.ddl-auto=update`

SPRING MVC

Si vous avez des questions, n'hésitez pas à nous contacter :

Département Informatique
UP ASI
Bureau E204